



INFO139 - Teoría de Autómatas

Simulador Máquina de Turing

Prof. María Eliana de la Maza

Integrantes:

Cristóbal Espinoza
Sebastián Pangué
Ninoska Toledo

1. Introducción

La **Máquina de Turing** es un modelo computacional teórico que permite definir y reconocer lenguajes formales. A diferencia de los autómatas finitos o de pila, cuenta con una cinta de memoria ilimitada y capacidad de lectura y escritura, lo que la convierte en el modelo más poderoso dentro de la **jerarquía de Chomsky**.

El presente trabajo consiste en el diseño e implementación de un simulador de Máquina de Turing reconocedora de lenguajes. El simulador permite al usuario definir una Máquina de Turing determinista ingresando sus componentes formales (estados, alfabetos, transiciones y símbolo blanco), para luego evaluar palabras de entrada y determinar si pertenecen o no al lenguaje reconocido por dicha máquina.

El programa fue implementado en Python y se distribuye como ejecutable (.exe) generado con **PyInstaller**, por lo que no requiere instalación de Python ni dependencias adicionales. Ofrece una interfaz gráfica desarrollada con **Tkinter** que permite definir la máquina y evaluar palabras de forma interactiva.

2. Desarrollo

El simulador está compuesto por dos módulos: **main.py**, que contiene la lógica de simulación, e **interfaz.py**, que implementa la interfaz gráfica.

2.1 Algoritmo de simulación

La función **simular_mt** recibe:

- Estado inicial
- Los estados de aceptación
- Las transiciones
- Palabra de entrada
- Símbolo blanco.

La cinta se representa como una lista de caracteres, donde la palabra de entrada se convierte símbolo por símbolo. Si la palabra es vacía, la cinta inicia con un símbolo blanco.

```
cinta = list(palabra_entrada) if palabra_entrada != "" else [simbolo_blanco]

cabezal = 0
estado_actual = estado_inicial
```

En cada paso, la máquina lee el símbolo bajo el cabezal y busca la transición correspondiente. Si existe, escribe el nuevo símbolo, cambia de estado y mueve el cabezal. Si no existe transición, la palabra es rechazada. La cinta se expande automáticamente hacia la derecha cuando el cabezal lo requiere.

```
if estado_actual in transiciones and simbolo_actual in transiciones[estado_actual]:
    nuevo_estado, simbolo_escrito, movimiento =
transiciones[estado_actual][simbolo_actual]
    cinta[cabezal] = simbolo_escrito
    estado_actual = nuevo_estado
    if movimiento == "D":
        cabezal += 1
    elif movimiento == "I": cabezal -= 1
```

Para detectar bucles infinitos, se registra cada configuración (cinta, posición del cabezal, estado actual) en un conjunto. Si una configuración se repite, la ejecución se aborta.

```
config = (cinta_str, cabezal, estado_actual)
if config in historial:
    return False, cinta_str.strip(simbolo_blanco)
historial.add(config)
```

2.2 Interfaz gráfica

La interfaz desarrollada con Tkinter organiza el uso en dos pestañas: la primera para definir la máquina (estados, alfabetos, transiciones) con validación inmediata de cada campo, y la segunda para evaluar palabras. Incluye un historial de palabras evaluadas y una visualización de la cinta final. También incorpora un ejemplo precargado del lenguaje **aⁿbⁿ**.

```
self.nb.add(self.tab_def, text=" 1 · Definir maquina ")
self.nb.add(self.tab_sim, text=" 2 · Evaluar palabras ")
```

3. Especificación del programa

Nombre: Simulador de Máquina de Turing

Requerimientos: No requiere instalación de Python ni dependencias adicionales. El programa es un ejecutable (.exe)

Cómo ejecutar: Hacer doble clic en **simulador_mt.exe**. La interfaz gráfica se abrirá directamente.

Ejemplo de funcionamiento: El programa incluye un ejemplo precargado del lenguaje **aⁿbⁿ** (cadenas con igual número de a seguidas de b). Al presionar "Cargar ejemplo" y luego "Construir máquina", se pueden evaluar palabras como ab, aabb, aaabbb (aceptadas) o aab, abb (rechazadas).

4. Lenguaje de Programación

El programa esta implementado en **Python**, utilizando únicamente estructuras de datos nativas del lenguaje. Se eligió por su facilidad para representar estructuras como diccionarios y conjuntos, y por ser un lenguaje fácil y claro de interpretar.

5. Estructuras de Datos

Se utilizaron las siguientes estructuras para almacenar cada componente de la Máquina de Turing:

Componente	Estructura	Justificación
Estado inicial	str	Un MT determinista tiene un único estado inicial.
Estado(s) final(es)	set	Permite representar uno o más estados de aceptación.
Transiciones	dict	Puede almacenar perfectamente las funciones de transición.
Palabra de entrada	str → list	La palabra se ingresa como str y se convierte a list para simular la cinta de un MT.

6. Conclusiones

El desarrollo de este simulador permitió aplicar de forma práctica los conceptos teóricos de las Máquinas de Turing vistos en clases. Se logró implementar un programa funcional capaz de simular cualquier Máquina de Turing determinista reconocedora de lenguajes, con validación de entradas y detección de bucles infinitos.

Como posibles mejoras, se podría agregar la visualización paso a paso de la ejecución (mostrando el estado de la cinta en cada transición) y la opción de guardar y cargar máquinas desde un archivo.

7. Bibliografía

Hopcroft, J., Motwani, R., & Ullman, J. (2007). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Pearson. <https://docencia.eafranco.com/materiales/teoriacomputacional/books/TeoriaDeAutomatas,lenguajesYComputacion-Hopcroft.pdf>

Python Software Foundation. (2024). *Python 3 Documentation*. <https://docs.python.org/es/3.14/>

PyInstaller Development Team. (2024). *PyInstaller Manual*. <https://pyinstaller.org/>